

Linked List - Complete Guide



Follow @techie_programmer for more Programming content!

Definition

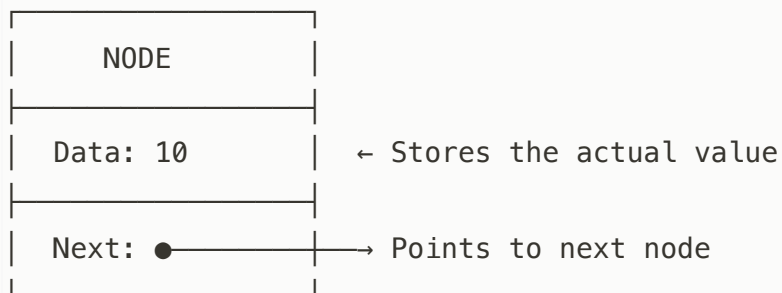
A **Linked List** is a linear data structure where elements are stored in nodes. Each node contains two parts:

- **Data:** The actual value stored
- **Next/Pointer:** Reference to the next node in the sequence

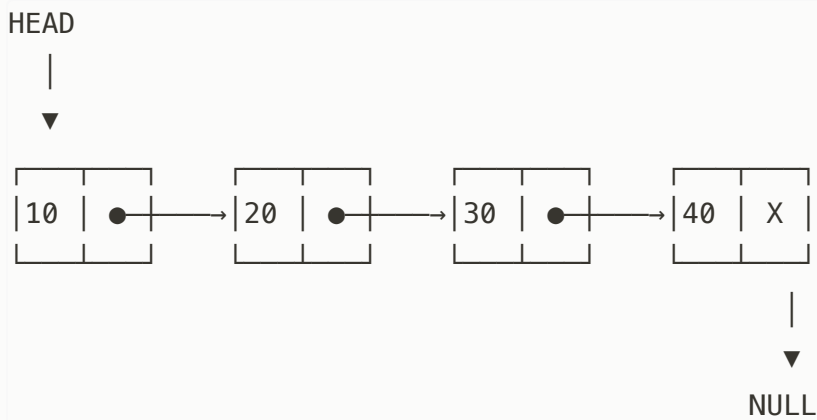
Unlike arrays, linked list elements are not stored in contiguous memory locations. Each element points to the next one, forming a chain.

Visual Structure

Node Structure



Linked List Representation

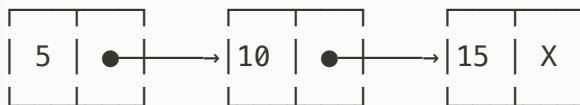


How It Works

Types of Linked Lists

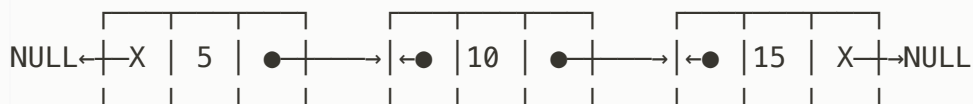
1. Singly Linked List

Each node points to the next node only.



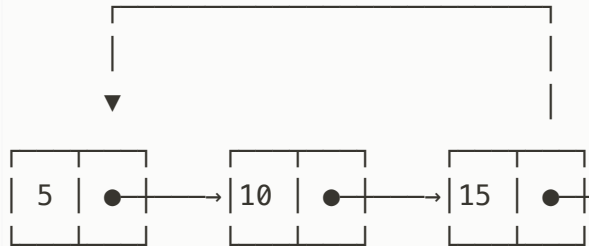
2. Doubly Linked List

Each node points to both next and previous nodes.



3. Circular Linked List

Last node points back to the first node.

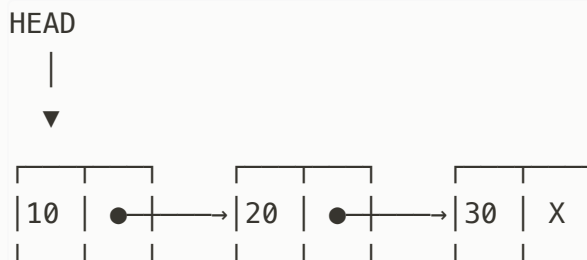


Basic Operations with Visualizations

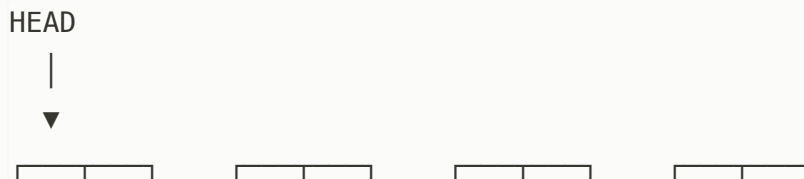
1. Insertion Operations

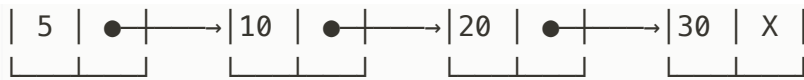
Insert at Beginning

Before:



After inserting 5:



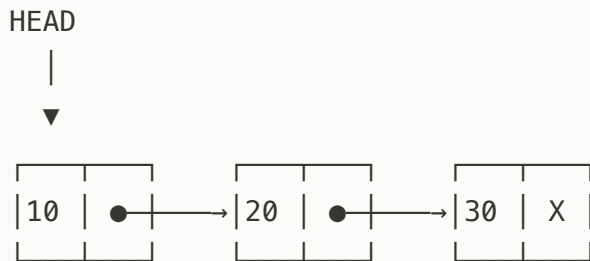


Steps:

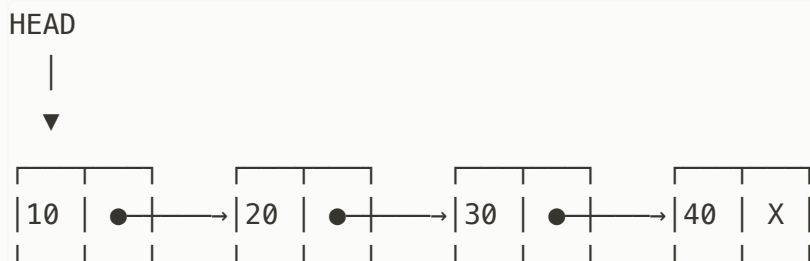
1. Create new node with data = 5
2. Point new node's next to current head
3. Update head to new node

Insert at End

Before:



After inserting 40:



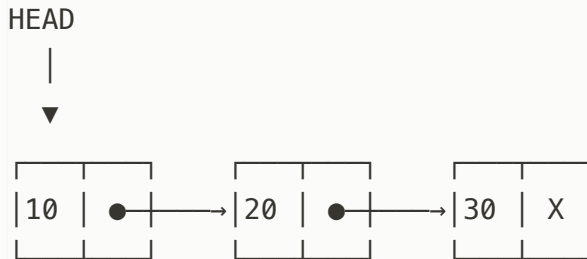
Steps:

1. Create new node with data = 40
2. Traverse to last node (where next = NULL)

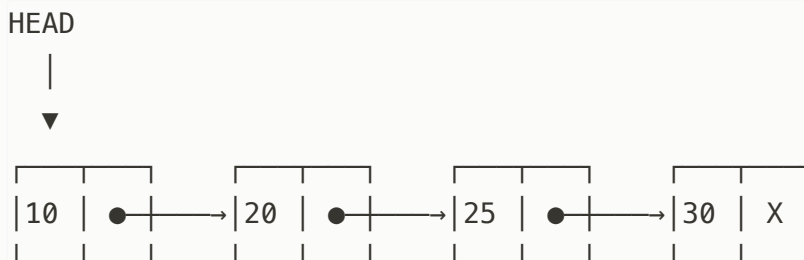
3. Point last node's next to new node

Insert at Position (Middle)

Before (Insert 25 at position 2):



After:



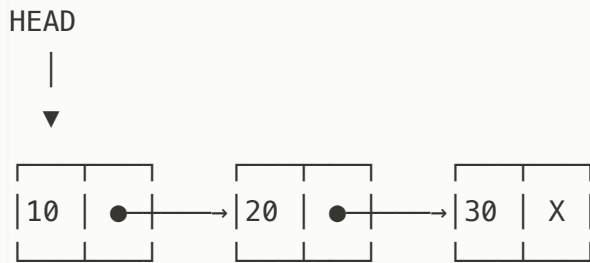
Steps:

1. Create new node with data = 25
2. Traverse to position 1 (node with 20)
3. Point new node's next to node 20's next
4. Point node 20's next to new node

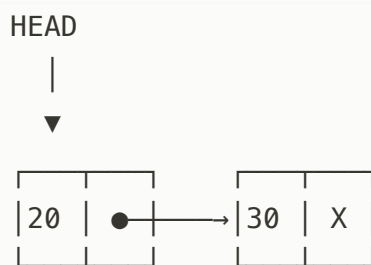
2. Deletion Operations

Delete from Beginning

Before:



After:

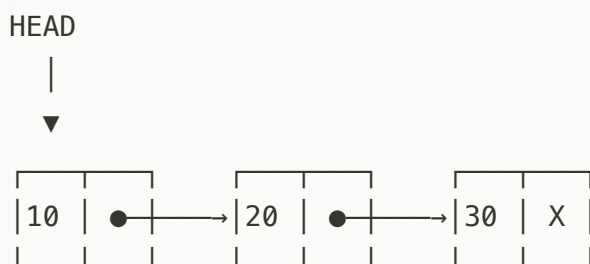


Steps:

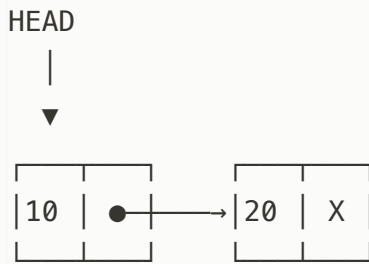
1. Store reference to current head
2. Move head to next node
3. Free the old head node

Delete from End

Before:



After:

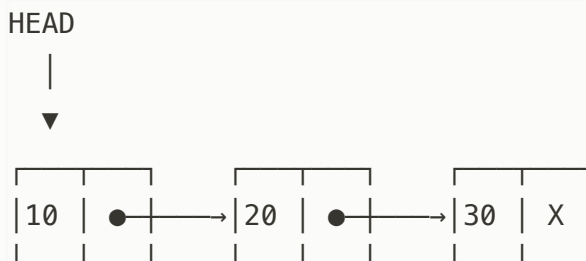


Steps:

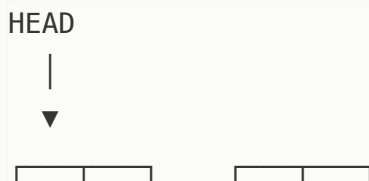
1. Traverse to second-last node
2. Set its next to NULL
3. Free the last node

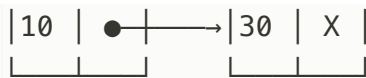
Delete Specific Value

Before (Delete node with value 20):



After:



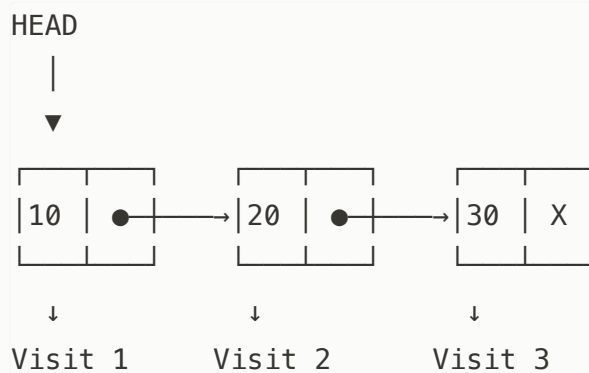


Steps:

1. Traverse to find node before target (10)
2. Point node 10's next to node 20's next (30)
3. Free node 20

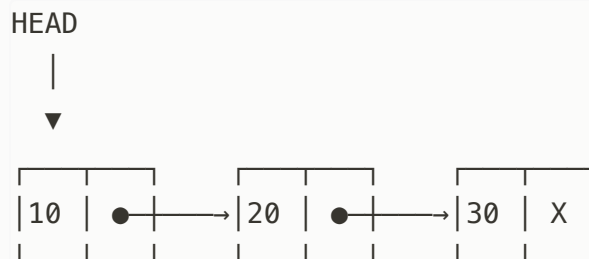
3. Traversal

Process: Visit each node from head to end



4. Search

Searching for value 20:



✗ ✓ FOUND!
Check Position 1

Time Complexity

Operation	Time Complexity	Space Complexity
Access	$O(n)$	$O(1)$
Search	$O(n)$	$O(1)$
Insertion (at beginning)	$O(1)$	$O(1)$
Insertion (at end)	$O(n)$	$O(1)$
Insertion (at position)	$O(n)$	$O(1)$
Deletion (at beginning)	$O(1)$	$O(1)$
Deletion (at end)	$O(n)$	$O(1)$
Deletion (at position)	$O(n)$	$O(1)$

Memory Representation

Array vs Linked List

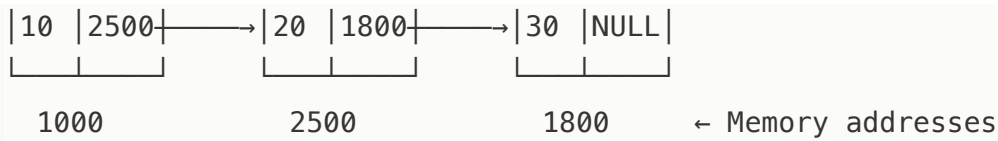
ARRAY (Contiguous Memory):

10	20	30	40	50
----	----	----	----	----

1000 1004 1008 1012 1016 ← Memory addresses

LINKED LIST (Non-contiguous Memory):





Code Implementations

Python Implementation

```
# Node class
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

# Linked List class
class LinkedList:
    def __init__(self):
        self.head = None

    # Insert at beginning
    def insert_at_beginning(self, data):
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node

    # Insert at end
    def insert_at_end(self, data):
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            return

        current = self.head
```

```

        while current.next:
            current = current.next
        current.next = new_node

# Insert at specific position
def insert_at_position(self, data, position):
    if position == 0:
        self.insert_at_beginning(data)
        return

    new_node = Node(data)
    current = self.head

    for i in range(position - 1):
        if current is None:
            print("Position out of bounds")
            return
        current = current.next

    new_node.next = current.next
    current.next = new_node

# Delete from beginning
def delete_from_beginning(self):
    if self.head is None:
        print("List is empty")
        return

    self.head = self.head.next

# Delete from end
def delete_from_end(self):
    if self.head is None:
        print("List is empty")
        return

    if self.head.next is None:

```

```

        self.head = None
        return

    current = self.head
    while current.next.next:
        current = current.next
    current.next = None

# Delete specific value
    def delete_value(self, value):
        if self.head is None:
            print("List is empty")
            return

        if self.head.data == value:
            self.head = self.head.next
            return

        current = self.head
        while current.next:
            if current.next.data == value:
                current.next = current.next.next
                return
            current = current.next

        print("Value not found")

# Search for a value
    def search(self, value):
        current = self.head
        position = 0

        while current:
            if current.data == value:
                return position
            current = current.next
            position += 1

```

```

        return -1

# Display the list
def display(self):
    if self.head is None:
        print("List is empty")
        return

    current = self.head
    while current:
        print(current.data, end=" -> ")
        current = current.next
    print("None")

# Get length
def length(self):
    count = 0
    current = self.head

    while current:
        count += 1
        current = current.next

    return count

# Example usage
if __name__ == "__main__":
    ll = LinkedList()

    # Insert elements
    ll.insert_at_end(10)
    ll.insert_at_end(20)
    ll.insert_at_end(30)
    ll.insert_at_beginning(5)
    ll.insert_at_position(15, 2)

```

```

print("Linked List:")
ll.display()

print(f"Length: {ll.length()}")

# Search
print(f"Search 20: Position {ll.search(20)}")

# Delete operations
ll.delete_from_beginning()
print("After deleting from beginning:")
ll.display()

ll.delete_from_end()
print("After deleting from end:")
ll.display()

ll.delete_value(20)
print("After deleting value 20:")
ll.display()

```

Java Implementation

```

// Node class
class Node {
    int data;
    Node next;

    Node(int data) {
        this.data = data;
        this.next = null;
    }
}

```

```

// LinkedList class
class LinkedList {
    Node head;

    LinkedList() {
        this.head = null;
    }

    // Insert at beginning
    public void insertAtBeginning(int data) {
        Node newNode = new Node(data);
        newNode.next = head;
        head = newNode;
    }

    // Insert at end
    public void insertAtEnd(int data) {
        Node newNode = new Node(data);

        if (head == null) {
            head = newNode;
            return;
        }

        Node current = head;
        while (current.next != null) {
            current = current.next;
        }
        current.next = newNode;
    }

    // Insert at specific position
    public void insertAtPosition(int data, int position) {
        if (position == 0) {
            insertAtBeginning(data);
            return;
        }
    }
}

```

```

Node newNode = new Node(data);
Node current = head;

for (int i = 0; i < position - 1; i++) {
    if (current == null) {
        System.out.println("Position out of bounds");
        return;
    }
    current = current.next;
}

newNode.next = current.next;
current.next = newNode;
}

// Delete from beginning
public void deleteFromBeginning() {
    if (head == null) {
        System.out.println("List is empty");
        return;
    }
    head = head.next;
}

// Delete from end
public void deleteFromEnd() {
    if (head == null) {
        System.out.println("List is empty");
        return;
    }

    if (head.next == null) {
        head = null;
        return;
    }

```



```

    Node current = head;
    while (current.next.next != null) {
        current = current.next;
    }
    current.next = null;
}

// Delete specific value
public void deleteValue(int value) {
    if (head == null) {
        System.out.println("List is empty");
        return;
    }

    if (head.data == value) {
        head = head.next;
        return;
    }

    Node current = head;
    while (current.next != null) {
        if (current.next.data == value) {
            current.next = current.next.next;
            return;
        }
        current = current.next;
    }

    System.out.println("Value not found");
}

// Search for a value
public int search(int value) {
    Node current = head;
    int position = 0;

    while (current != null) {

```

```

        if (current.data == value) {
            return position;
        }
        current = current.next;
        position++;
    }

    return -1;
}

// Display the list
public void display() {
    if (head == null) {
        System.out.println("List is empty");
        return;
    }

    Node current = head;
    while (current != null) {
        System.out.print(current.data + " -> ");
        current = current.next;
    }
    System.out.println("null");
}

// Get length
public int length() {
    int count = 0;
    Node current = head;

    while (current != null) {
        count++;
        current = current.next;
    }

    return count;
}

```

```

// Main method for testing
public static void main(String[] args) {
    LinkedList ll = new LinkedList();

    // Insert elements
    ll.insertAtEnd(10);
    ll.insertAtEnd(20);
    ll.insertAtEnd(30);
    ll.insertAtBeginning(5);
    ll.insertAtPosition(15, 2);

    System.out.println("Linked List:");
    ll.display();

    System.out.println("Length: " + ll.length());

    // Search
    System.out.println("Search 20: Position " +
ll.search(20));

    // Delete operations
    ll.deleteFromBeginning();
    System.out.println("After deleting from beginning:");
    ll.display();

    ll.deleteFromEnd();
    System.out.println("After deleting from end:");
    ll.display();

    ll.deleteValue(20);
    System.out.println("After deleting value 20:");
    ll.display();
}
}

```

C Implementation

```
#include <stdio.h>
#include <stdlib.h>

// Node structure
struct Node {
    int data;
    struct Node* next;
};

// Create a new node
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct
Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

// Insert at beginning
void insertAtBeginning(struct Node** head, int data) {
    struct Node* newNode = createNode(data);
    newNode->next = *head;
    *head = newNode;
}

// Insert at end
void insertAtEnd(struct Node** head, int data) {
    struct Node* newNode = createNode(data);

    if (*head == NULL) {
        *head = newNode;
        return;
    }
}
```

```

    struct Node* current = *head;
    while (current->next != NULL) {
        current = current->next;
    }
    current->next = newNode;
}

// Insert at specific position
void insertAtPosition(struct Node** head, int data, int position)
{
    if (position == 0) {
        insertAtBeginning(head, data);
        return;
    }

    struct Node* newNode = createNode(data);
    struct Node* current = *head;

    for (int i = 0; i < position - 1; i++) {
        if (current == NULL) {
            printf("Position out of bounds\n");
            free(newNode);
            return;
        }
        current = current->next;
    }

    newNode->next = current->next;
    current->next = newNode;
}

// Delete from beginning
void deleteFromBeginning(struct Node** head) {
    if (*head == NULL) {
        printf("List is empty\n");
        return;
    }

```

```

    }

    struct Node* temp = *head;
    *head = (*head)->next;
    free(temp);
}

// Delete from end
void deleteFromEnd(struct Node** head) {
    if (*head == NULL) {
        printf("List is empty\n");
        return;
    }

    if ((*head)->next == NULL) {
        free(*head);
        *head = NULL;
        return;
    }

    struct Node* current = *head;
    while (current->next->next != NULL) {
        current = current->next;
    }

    free(current->next);
    current->next = NULL;
}

// Delete specific value
void deleteValue(struct Node** head, int value) {
    if (*head == NULL) {
        printf("List is empty\n");
        return;
    }

    if ((*head)->data == value) {

```

```

        struct Node* temp = *head;
        *head = (*head)->next;
        free(temp);
        return;
    }

    struct Node* current = *head;
    while (current->next != NULL) {
        if (current->next->data == value) {
            struct Node* temp = current->next;
            current->next = current->next->next;
            free(temp);
            return;
        }
        current = current->next;
    }

    printf("Value not found\n");
}

// Search for a value
int search(struct Node* head, int value) {
    struct Node* current = head;
    int position = 0;

    while (current != NULL) {
        if (current->data == value) {
            return position;
        }
        current = current->next;
        position++;
    }

    return -1;
}

// Display the list

```

```

void display(struct Node* head) {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }

    struct Node* current = head;
    while (current != NULL) {
        printf("%d -> ", current->data);
        current = current->next;
    }
    printf("NULL\n");
}

```

```

// Get length
int length(struct Node* head) {
    int count = 0;
    struct Node* current = head;

    while (current != NULL) {
        count++;
        current = current->next;
    }

    return count;
}

```

```

// Free the entire list
void freeList(struct Node** head) {
    struct Node* current = *head;
    struct Node* next;

    while (current != NULL) {
        next = current->next;
        free(current);
        current = next;
    }
}

```



```

    *head = NULL;
}

// Main function for testing
int main() {
    struct Node* head = NULL;

    // Insert elements
    insertAtEnd(&head, 10);
    insertAtEnd(&head, 20);
    insertAtEnd(&head, 30);
    insertAtBeginning(&head, 5);
    insertAtPosition(&head, 15, 2);

    printf("Linked List:\n");
    display(head);

    printf("Length: %d\n", length(head));

    // Search
    printf("Search 20: Position %d\n", search(head, 20));

    // Delete operations
    deleteFromBeginning(&head);
    printf("After deleting from beginning:\n");
    display(head);

    deleteFromEnd(&head);
    printf("After deleting from end:\n");
    display(head);

    deleteValue(&head, 20);
    printf("After deleting value 20:\n");
    display(head);

    // Free memory

```

```
    freeList(&head);  
  
    return 0;  
}
```

Key Advantages

1. **Dynamic Size:** Can grow or shrink during execution
2. **Efficient Insertion/Deletion:** $O(1)$ at beginning
3. **No Memory Waste:** Allocates memory as needed
4. **Easy Implementation:** Simple pointer manipulation

Key Disadvantages

1. **Sequential Access:** Cannot directly access elements by index
2. **Extra Memory:** Requires additional memory for pointers
3. **No Random Access:** Must traverse from beginning to reach any element
4. **Cache Performance:** Poor cache locality compared to arrays